PHY 112L
Entropy of a 2-D Ideal-Gas

Michael Mulhearn

November 17, 2025

1 Introduction

In this assignment, you will introduce the Gibbs Entropy as a convenient way to measure changes in entropy for a numerical simulation. We will apply it to measure the change of entropy of an ideal gas due to heating and free expansion in a molecular simulation, and compare it to our expectation. We will also examine a gas expanding into a region at a higher potential energy.

2 Gibbs Entropy

The entropy of a macrostate is defined as:

$$S = k \ln \Omega \tag{1}$$

where k is Boltzmann's constant and Ω is the multiplicity of microstates corresponding to the macrostate.

To compute the Gibbs Entropy, we divide the total phase space into M cells. The macrostate is specified by the number of particles in each cell n_i. To count these microstates, start by assuming the we have only two cells, with the first containing a particles. The binomial theorem gives us the multiplicity of states:

$$\Omega_{(2)} = \frac{N!}{a!(N-a)!}$$

If we divide the second cell into two cells containing b and c particles respectively, with:

$$N - a = b + c$$

then the binomial theorem implies that the multiplicity of states is:

$$\Omega_{(3)} = \frac{N!}{a!(N-a)!} \cdot \frac{(N-a)!}{b!c!} = \frac{N!}{a!b!c!}$$

If we continue in this way for M cells, we arrive at the multinomial coefficient:

$$\Omega = \frac{N!}{\prod_{i=1}^{M} n_i!} \tag{2}$$

Calculating

$$\ln \Omega = \ln N! - \sum_i \ln n_i!$$

and applying Stirling's approximation:

$$\ln \Omega = N \ln N - N - \sum_i (n_i \ln n_i - n_i)$$

Using $\sum_i n_i = N$ we expand the sum and cancel terms:

$$\ln \Omega = N \ln N - N - \sum_i n_i \ln n_i + \sum_i n_i = N \ln N - \sum_i n_i \ln n_i,$$

and hence

$$\ln \Omega = -\sum_i n_i \big(\ln n_i - \ln N \big) = -\sum_i n_i \ln\Big(\frac{n_i}{N}\Big).$$

and so:

$$\frac{S}{kN} = -\sum_i \frac{n_i}{N} \ln\left(\frac{n_i}{N}\right)$$

and defining probability per bin as:

$$p_i = \frac{n_i}{N}$$

we have at last:

$$\frac{S}{kN} = -\sum_i p_i \ln(p_i) \tag{3}$$

which is known as the Gibbs Entropy (or Shannon Entropy for k=1). When calculating the Gibbs Entropy numerically, set the contribution from bins with $p_i = 0$ to zero, which is consistent with the limiting behavior.

For a uniform distribution in M cells, the probability of visiting each cell is identical, with:

$$p_i = 1/M$$

and so in this case:

$$\frac{S}{kN} = \ln(M) \tag{4}$$

Notice that even in this simple case, the Gibbs Entropy is dependent on the choice of bins.

As is always the case for classical entropy, the Gibbs entropy is only determined to within an overall constant, which in this case depends on the cells that we have chosen. However, as long as we keep our choice of bins constant, we can reliably calculate a change in entropy between initial and final states using this formula. To compute p_i, we need only count the number of particles present in each cell and divide by the total number of particles.

To determine the values of p_i more reliably (with more statistics) we can simulate many time steps, and count the number of visits of particles to each cell. Dividing the result by the total number of visits (the number of particles times the number of time steps) we determine p_i in each cell with less statistical uncertainty. This only works once the system has reached equilibrium (or else the Gibbs Entropy will be changing).

3 Entropy Changes for an Ideal Gas

If no work is done on our gas, then the heat added equals the change in internal energy,

$$\Delta U = Q.$$

For a reversible heating process the entropy change is

$$\mathrm{d}S = \frac{\delta Q_{\mathrm{rev}}}{T}.$$

At constant volume (no work), $\delta Q_{\mathrm{rev}} = \mathrm{d}U$, and for a 2-D ideal gas $U = NkT$, so $\mathrm{d}U = Nk\,\mathrm{d}T$. Hence

$$\mathrm{d}S = \frac{Nk\,\mathrm{d}T}{T},$$

and integrating between initial and final temperatures gives

$$\Delta S = \int_{T_i}^{T_f} \frac{Nk\,\mathrm{d}T}{T} = Nk \ln\left(\frac{T_f}{T_i}\right).$$

Next, suppose that instead that we expand the gas from V_1 to V_2 at constant temperature. The velocity profile remains unchanged, due to the constant temperature, and so the phase-space only changes by the volume, with:

$$\Omega \propto V^N$$

and so:

$$\begin{aligned} \Delta S &= k\,\ln(\Omega_2) - k\,\ln(\Omega_1) \\ &= k \ln\left(\frac{\Omega_2}{\Omega_1}\right) \end{aligned}$$

and so:

$$\Delta S = N\,k\,\ln\left(\frac{V_2}{V_1}\right) \tag{5}$$

4 Ideal Gas with a Step Potential

Consider an ideal gas contained within $0 \leq y \leq 4a$ subject to a potential energy:

$$V(y) = \begin{cases} V_{\mathrm{step}} & y > 2a \\ 0 & \text{otherwise} \end{cases} \tag{6}$$

We can develop an accurate numerical simulation for this potential based on conservation of energy, but with the crucial caveat that the molecules momentum in the x direction is unchanged by the potential. We define the contribution to the kinetic energy due to the particles' motion in the y-direction as

$$K_{(y)} \equiv \frac{1}{2} m v_y^2 \tag{7}$$

Molecules in the lower region ($y < 2a$) with:

$$K_{(y)} \leq V_{\mathrm{step}}$$

never escape the lower region, and bounce at the boundaries. But molecules with

$$K_{(y)} > V_{\mathrm{step}}$$

can cross from the lower region to the upper region ($y > 2a$), and have the momentum in the y direction reduced such that:

$$K_{(y)} \to K_{(y)} - V_{\text{step}}$$

Molecules can always cross from the upper region into the lower region, and have their momentum in the y direction increased such that:

$$K_{(y)} \to K_{(y)} + V_{\text{step}}$$

We will call the function that implements this algorithm `reflect`.

The relative occupancy of the upper and lower regions (which are the same area) will be:

$$\frac{p_{\text{up}}}{p_{\text{dn}}} = \exp\left(-\frac{V_{\text{step}}}{kT}\right) \equiv \alpha$$

and so:

$$p_{\text{up}} = \frac{\alpha}{1+\alpha} \tag{8}$$

and

$$p_{\text{dn}} = \frac{1}{1+\alpha} \tag{9}$$

If we consider this like a two state system going from the ground state ($S = 0$) to a state at equilibrium with occupancies p_{up} and p_{dn}, we expect the change in entropy to be:

$$\frac{\Delta S}{kN} = -(p_{\text{up}} \ln p_{\text{up}} + p_{\text{dn}} \ln p_{\text{dn}})$$

5 System of Units

We will use the same system of units as the one we use for the first assignment (as there is no longer any need to add a constraint for gravity, which we will not consider in this assignment). We assume there is some relevant length scale a which we set to 1 and a relevant temperature T_0 which we set to 1 as well.

The other units that we defined in the previous homework continue to apply, so let's recall them here. Motivated by the Maxwell-Boltzmann distribution, the velocity unit v_0 is defined by:

$$v_0^2 \equiv \frac{kT_0}{m}$$

and the unit for time is:

$$\tau \equiv a/v_0$$

Motivated by the ideal gas law, we can set the unit of total energy to:

$$U_0 = NkT_0$$

For this assignment, you will measure the potential energy (of a single particle) using units of

$$V_0 = kT_0$$

Remember that for analytic work, we keep these factors in place. But when using an analytic result within our computer programs, we set all of these quantities to 1:

$$m = a = T_0 = v_0 = \tau = U_0 = V_0 = 1$$

As before, the temperature of an ideal gas is still calculated through the average kinetic energy, which in our system of units becomes:

$$\frac{T}{T_0} = \frac{\langle v_x^2/v_0^2\rangle + \langle v_y^2/v_0^2\rangle}{2} \tag{10}$$

Equation (10) gives the dimensionless temperature, which in our units is also equal to the total kinetic energy of the gas divided by U_0:

$$\frac{U_{\text{kinetic}}}{U_0} = \frac{T}{T_0}.$$

(Equivalently, this is the average energy per particle, in dimensionless units.)

6 Required Functions

You must implement the following functions as part of your solution. You may substitute similar signatures if using a language other than Python, but you must use exactly these signatures if using Python. You do not need to include the docstrings. You are encouraged to define additional functions as part of your solution, but these functions are the most significant.

```python
def gibbs_entropy(*, vx, vy, y, hbins, vmax, vbins):
    """Compute dimensionless Gibbs entropy S/(kN).

    Parameters:
        vx    : np.ndarray  x-velocity components.
        vy    : np.ndarray  y-velocity components.
        y     : np.ndarray  Vertical positions.
        hbins : int         Number of y bins.
        vmax  : float       Max |v| for velocity range (-vmax, vmax).
        vbins : int         Number of velocity bins per component.
    Returns:
        float The dimensionless Gibbs entropy S/(kN)
    """
    pass
```

```python
def collide_reservoir(*, vx, vy, Tr, ncoll):
    """Perform collisions of selected particles with a gas reservoir.

    Parameters:
        vx    : np.ndarray  Particle x-velocities (updated in place).
        vy    : np.ndarray  Particle y-velocities (updated in place).
        Tr    : float       Gas reservoir temperature.
        ncoll : int         Number of collisions to perform.
    """
    pass
```

```python
def step(*, vy, y, dt):
    """Advance particle positions for one timestep.

    Parameters:
        vy : np.ndarray  Vertical velocity components.
        y  : np.ndarray  Vertical positions (updated in place).
        dt : float       Time step.
    """
    pass
```

```python
def bounce(*, vy, y, h):
    """Reflect particles elastically at y=0 and y=h boundaries.

    Parameters:
        vy : np.ndarray  Vertical velocity components (updated in place).
        y  : np.ndarray  Vertical positions (updated in place).
        h  : float       Container height.
    """
    pass
```

```python
def reflect(*, vy, y_old, y, h_step, V_step):
    """Handle reflection and transmission at a potential step.

    Parameters:
        vy     : np.ndarray  Vertical velocities (updated in place).
        y_old  : np.ndarray  Positions before the time step.
        y      : np.ndarray  Positions after the time step (updated in place).
        h_step : float       The y-position of the potential step.
        V_step : float       Potential of the upper region
    """
    pass
```

These functions use the Keyword-only arguments feature of python. The arguments following the asterix (*) must be assigned by name, like `vx=vx_a` or `vx=vx`. This is a good practice when you have arguments of identical types and no obvious order.

7 Homework Problems

Zero credit if you do not use our system of units consistently! When I ask you to calculate a quantity, e.g. temperature, I want the answer in our units, e.g. T_0. Do not first calculate in SI units and then convert!

As a starting point, as in previous assignments, you will want to initialize a numerical model for an ideal gas tracking variables v_x, v_y, and y for $N = 10000$ molecules, with height $H = 4a$.

Problem 1: Implement the function `gibbs_entropy`, with the required signature, that computes the Gibbs Entropy for a gas using the specified bins. Validate your function on an initial distribution that is uniformly distributed in v_x, v_y and h.

Problem 2: Implement the function `collide_reservoir`, with the required signature, that simulates collisions of your gas with a thermal reservoir. Calculate the change in entropy when your gas is heated from $T = 1$ to $T = 2$ and compare to your expectation. Remember that for the Gibbs Entropy will change by an (unknown) constant factor if you change the binning, so you must keep the bins **exactly the same** for both the initial and final state in order to accurately determine the change in entropy from your simulation.

Problem 3: Implement the functions `step` and `bounce`. Use them to simulate the free expansion of a gas from $h = 2a$ to $h = 4a$ at a constant temperature $T_i = 1$. Compare your measured change in entropy to your expectation.

Problem 4: Implement the function `reflect`. Use it to simulate the expansion of a gas from $h = 2a$ to $h = 4a$ at a constant temperature $T_i = 1$ when the upper region from $h = 2a$ to $h = 4a$ is at a higher potential energy V_{step} compared to lower region, which you should take as the zero of potential. Show that the occupancy of the upper region matches your expectation.

Problem 5: Calculate the change in entropy from Problem 4 and compare to your expectation.